

DAY 18

Document Chunking in Depth

GPT + AI Agents + RAG + n8n + API Integration + Business Automation

Duration	60 minutes
Level	Beginner to Intermediate
Main Project	Chunking Lab for a Company Knowledge RAG System
Core Transition	From "we can search vectors" to "we control what each searchable knowledge unit contains"

Day 18 Core Question

What is the **right unit of knowledge** to embed and retrieve so that the model gets enough context to answer correctly without drowning in irrelevant text?

Prepared for the 30-Day AI Automation Lecture Series

Table of Contents

1. Learning Goals & Schedule
2. Day 17 Review
3. What Is Document Chunking?
4. Why Chunking Quality Controls RAG
5. Chunk Size
6. Chunk Overlap
7. Fixed-Size & Recursive Chunking
8. Structure-Aware & Semantic Chunking
9. Parent-Child, Small-to-Big & Hierarchical Retrieval
10. Metadata & Context Preservation
11. Special Content: Tables, Code, Lists, PDFs
12. Query-Aware and Advanced Chunking Patterns
13. OpenAI Vector Store Chunking
14. Practical Python Chunking Lab
15. Chunking Evaluation & Failure Modes
16. Production Design & Business Use Cases
17. Exercise, Quiz, Homework & Interview Prep
18. Day 19 Preview & References

Teaching Note

Chunking is one of the highest-leverage RAG design choices. Students should leave Day 18 understanding that a vector store cannot retrieve information that was split, merged, parsed, or labeled badly during ingestion.

1. Learning Goals & 60-Minute Schedule

Day 18 focuses on the transformation between parsed documents and embeddings. The goal is not to memorize one magic chunk size. The goal is to understand how chunk boundaries affect recall, precision, context quality, cost, citations, and user trust.

Learning Objectives

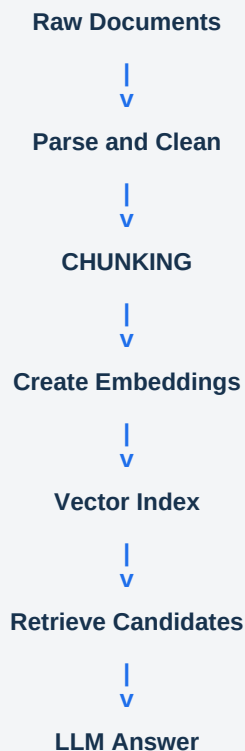
- Explain why long documents are divided into retrieval units before embedding.
- Distinguish chunk size from chunk overlap.
- Explain the tradeoff between small, precise chunks and large, context-rich chunks.
- Compare fixed-token, character, recursive, sentence, paragraph, heading, semantic, and hierarchical chunking.
- Design metadata that preserves source, section, page, policy version, department, and access scope.
- Handle tables, code, lists, FAQs, contracts, and scanned/PDF-derived content more carefully.
- Explain parent-child and small-to-big retrieval.
- Build a simple Python chunking lab and compare strategies.
- Evaluate chunking using retrieval tests rather than intuition alone.

Time	Topic
0-5	Day 17 review
5-10	What chunking is
10-18	Chunk size and overlap
18-28	Fixed, recursive, and structure-aware splitting
28-36	Semantic and hierarchical strategies
36-43	Metadata and special content
43-49	OpenAI vector store chunking
49-55	Python chunking lab
55-58	Evaluation and failure modes
58-60	Quiz + homework

2. Day 17 Review: Where Chunking Fits

Day 17 showed how embeddings make semantic similarity searchable. But the embedding is created for a specific piece of text. Day 18 asks: **what should that piece of text be?**

RAG ingestion-to-answer pipeline



Key Dependency

Embedding quality cannot repair a chunk that combines unrelated topics, loses a critical heading, separates a question from its answer, or omits the table row containing the required fact.

Day 17 vs Day 18

Day 17	Day 18
How do we represent meaning?	What text unit should we represent?
How do vectors become searchable?	Where should document boundaries be placed?
Similarity and nearest neighbors	Context preservation and retrieval granularity
Top-K and ranking	Chunk size, overlap, metadata, hierarchy

3. What Is Document Chunking?

Chunking is the process of dividing source content into smaller units that can be embedded, indexed, retrieved, and supplied to a model.

Simple Example

EMPLOYEE HANDBOOK

Section 1 - Vacation

Employees receive 20 vacation days per year.
Unused days may carry over up to 5 days.

Section 2 - Expenses

Expense claims must be submitted within 30 days.
Receipts are required for expenses over \$25.

A basic section-aware chunker might create:

Chunk 1

Title: Vacation

Employees receive 20 vacation days per year.
Unused days may carry over up to 5 days.

Chunk 2

Title: Expenses

Expense claims must be submitted within 30 days.
Receipts are required for expenses over \$25.

Important

The vector database does not search an abstract "whole handbook" in one step. It usually searches many indexed units derived from the handbook.

Chunking Is a Data-Modeling Decision

- What should stay together?
- What can safely be separated?
- How much local context should each result contain?
- What metadata should accompany each chunk?
- How will a retrieved chunk be cited back to the source?

4. Why Chunking Quality Controls RAG

A RAG system can have a strong model, a strong embedding model, and a fast vector store and still fail because the retrieved units were designed poorly.

Four Ways Chunking Changes Retrieval

Dimension	Chunking Impact
Recall	Was the needed fact preserved inside a retrievable unit?
Precision	Does the chunk contain mostly relevant content or lots of unrelated text?
Context	Does the chunk preserve definitions, headings, exceptions, and surrounding meaning?
Cost / latency	How many chunks and tokens must be stored, searched, reranked, and sent to the LLM?

Failure Example: Too Large

A 2,500-token chunk contains vacation, expenses, security, travel, and parental-leave policies. A query about expense receipts may retrieve the chunk, but most of its content is noise.

Failure Example: Too Small

One chunk says "The limit is 5 days" while the previous chunk contains the heading "Vacation carryover." The retrieved chunk alone may be ambiguous.

RAG Design Principle

The target is not the smallest chunk or the largest chunk. The target is a chunk that is **semantically coherent, independently understandable, and useful for answering likely questions.**

5. Chunk Size: The Main Granularity Tradeoff

Chunk size is the approximate amount of content placed into each searchable unit. It may be measured in characters, words, sentences, paragraphs, or tokens.

Small Chunks

Advantages	Risks
• More precise matches • Less irrelevant text in each hit • Often easier to cite exact evidence • More granular filtering	• May lose surrounding context • More vectors and index entries • More top-K slots needed to reconstruct an answer • Definitions and exceptions can be separated

Large Chunks

Advantages	Risks
• Preserve local context • Fewer vectors • Policies and exceptions more likely to remain together • Often better for narrative explanations	• Semantic signal can be diluted • More irrelevant text reaches the LLM • Higher context-token usage per hit • One result may mix several topics

Illustrative Starting Points - Not Universal Rules

Content Type	Possible Starting Unit	Reason
FAQ	One Q+A pair	Naturally atomic
Policy manual	One subsection or several paragraphs	Keep rule + exception together
Product catalog	One product / variant record	Strong entity boundary
Contracts	Clause / subsection	Preserve legal structure
Source code	Function / class / logical block	Preserve symbols and dependencies
Meeting transcript	Speaker/topic windows	Keep conversational context

Do Not Memorize a Magic Number

A good chunk size depends on document structure, query type, model context, embedding behavior, retrieval strategy, reranking, and the downstream answer task.

6. Chunk Overlap: Preserving Boundary Context

Overlap repeats some content from the end of one chunk at the beginning of the next. It is mainly used to reduce boundary failures.

No Overlap

Chunk A: Employees may carry over unused vacation...
Chunk B: ...up to five days into the next calendar year.

If the split falls in the middle of the rule, neither chunk may be ideal.

With Overlap

Chunk A: Employees may carry over unused vacation up to five days...
Chunk B: ...carry over unused vacation up to five days into the next calendar year.

Benefits and Costs

More Overlap	Less Overlap
More boundary continuity	Less duplicate indexed text
Higher chance complete phrase appears	Lower storage / embedding cost
May improve recall	May improve diversity in top-K
Can produce near-duplicate search results	Higher risk of losing context at boundaries

Overlap Is Insurance, Not a Substitute for Structure

If your source already has strong headings, paragraphs, clauses, or FAQ boundaries, structure-aware chunking can reduce the need for heavy overlap.

Duplicate Retrieval Problem

Large overlap can cause top-5 retrieval to return five nearly identical chunks. This wastes ranking slots. Deduplication, parent grouping, or smaller overlap can help.

7. Fixed-Size and Recursive Chunking

Strategy A - Fixed Character / Token Windows

Split every N characters or tokens, optionally with overlap.

```
chunk_size = 800
overlap = 120

start = 0
while start < len(tokens):
    end = start + chunk_size
    chunks.append(tokens[start:end])
    start = end - overlap
```

Strengths

- Simple, deterministic, fast, and easy to benchmark.
- Useful as a baseline and for content with weak structure.

Weaknesses

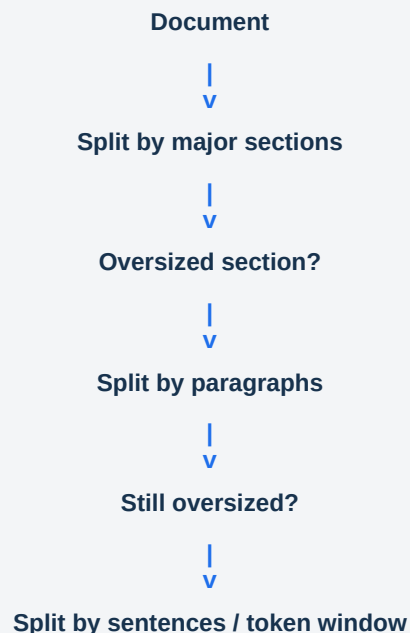
- Can split headings from content, sentences in half, or tables at bad boundaries.

Strategy B - Recursive Splitting

Try natural separators first and use smaller separators only when a block is still too large.

```
Preferred separators:
1. Section / heading boundaries
2. Blank lines
3. Paragraphs
4. Sentences
5. Words / tokens as last resort
```

Recursive splitting idea



Why Recursive Splitting Is Popular

It combines predictable size constraints with an effort to preserve natural language boundaries.

8. Structure-Aware and Semantic Chunking

Structure-Aware Chunking

Use the document's existing logical structure: titles, headings, clauses, bullets, rows, speaker turns, Markdown headers, HTML sections, or code symbols.

```
Document
  H1: Leave Policy
    H2: Vacation
      Paragraphs...
    H2: Sick Leave
      Paragraphs...
  H1: Expenses
    H2: Receipts
      Paragraphs...
```

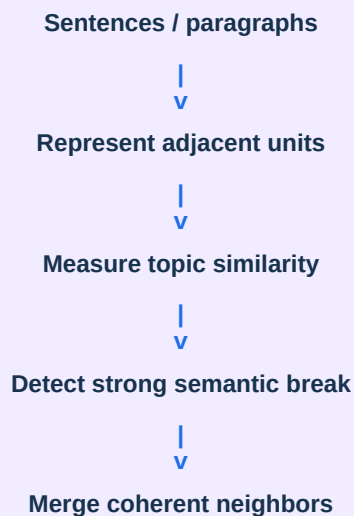
A structure-aware chunk can preserve the heading path as both text and metadata:

```
heading_path = "Leave Policy > Vacation"
chunk_text = "Vacation: Employees receive 20 days..."
```

Semantic Chunking

Semantic chunking attempts to create boundaries where the topic meaning changes, rather than only where a character/token counter reaches a limit.

Semantic boundary concept



Potential Advantages

- Better topic coherence for mixed documents.
- Can adapt to sections that are naturally short or long.

Tradeoffs

- More preprocessing complexity and cost.
- Boundary algorithm becomes another component that needs evaluation.
- Semantic similarity is not the same as business structure.

Professional Rule

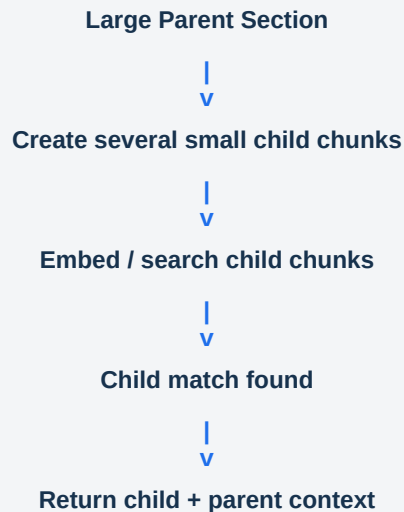
Use semantic chunking when it solves a measured retrieval problem - not simply because it sounds more advanced.

9. Parent-Child, Small-to-Big, and Hierarchical Retrieval

Sometimes the best unit for **search** is smaller than the best unit for **answer context**. Hierarchical techniques separate these jobs.

Parent-Child Pattern

Small search unit, larger context unit



Example

Parent: Section 4 - Expense Reimbursement (1,500 tokens)

Child 1: receipt requirements
Child 2: submission deadline
Child 3: mileage reimbursement
Child 4: international travel

Search can identify Child 1 precisely, while the generation step can receive Child 1 plus a larger parent excerpt containing exceptions and definitions.

Small-to-Big Retrieval

Index compact units for precise semantic matching, then expand around a match to include neighboring sentences, the full paragraph, or the enclosing section.

Hierarchical Index

Level	Example	Use
Document	Employee Handbook	Broad routing / corpus selection
Section	Expense Reimbursement	Topic-level retrieval
Chunk	Receipt requirement paragraph	Precise evidence
Sentence / row	Receipt required above \$25	Exact fact

Why This Matters

A single universal chunk size forces one unit to satisfy both search precision and answer-context completeness. Hierarchical retrieval lets those goals be optimized separately.

10. Metadata and Context Preservation

Chunk text is only part of a retrieval record. Metadata helps filter, organize, cite, secure, and interpret that text.

Useful Metadata Fields

Field	Example	Why Useful
document_id	handbook_2026	Group chunks from one source
title	Employee Handbook	Human-readable source
section	Vacation Carryover	Restore topic context
page	42	Citations / audit
department	HR	Filtering
region	US	Policy scoping
version	2026-07	Freshness
effective_date	2026-07-01	Resolve outdated policies
access_scope	employees	Authorization-aware retrieval

Context Prefixing

Sometimes a chunk is clearer if the heading is included directly in the embedded text.

Bad standalone chunk:

"Up to 5 days may carry over into the next calendar year."

Better:

"Vacation Carryover - Up to 5 unused vacation days may carry over into the next calendar year."

Metadata vs Embedded Text

Use Metadata	Use Chunk Text
Exact filters and authorization	Semantic meaning needed for search
Dates, department, IDs, region	Heading / topic wording
Source tracing	Definitions required to interpret the passage
Version control	Question-answer language

Security Note

Access control should be enforced by trusted application/database filters. Do not rely on the LLM to remember which confidential chunks a user is allowed to retrieve.

11. Special Content: Tables, Code, Lists, and PDFs

Uniform paragraph chunking often fails on structured or non-prose content. Production RAG should treat content type as part of ingestion design.

Tables

A table row without headers may become meaningless.

```
Weak chunk:
"Enterprise | $499 | Unlimited | Yes"

Better chunk:
"Plan comparison. Plan=Enterprise; Price=$499; Users=Unlimited; SS0=Yes."
```

For large tables, consider row-level records with repeated header context, grouped rows, or a structured database/query tool instead of RAG.

Lists and Procedures

Keep ordered steps together when the order matters. Splitting step 4 from steps 1-3 can produce unsafe or incomplete instructions.

Source Code

Prefer logical symbols such as function, class, method, or module boundaries. Include signature/import or parent symbol metadata where useful.

```
metadata = {
  "file": "billing.py",
  "symbol": "calculate_refund",
  "type": "function",
  "language": "python"
}
```

PDFs

PDFs are layout containers. Parsed reading order can be wrong, headers/footers may repeat, multi-column text can interleave, and tables can flatten badly.

PDF Problem	Mitigation
Repeated header/footer	Remove boilerplate before chunking
Two-column text	Use layout-aware parser
Page number splits sentence	Merge text using structure, not raw page boundaries
Scanned page	OCR only when needed and verify extraction
Table flattened	Extract table structurally or serialize headers + rows

12. Query-Aware and Advanced Chunking Patterns

FAQ / Q+A Chunking

Keep each question with its answer. If there are alternate question phrasings, they can be added to the chunk or used for retrieval augmentation.

Clause-Aware Contracts

Use clause/subclause boundaries and retain definitions or references needed to interpret the clause. Pure fixed windows may separate exceptions from the rule they modify.

Conversation / Transcript Chunking

Possible units include topic segments, speaker-turn windows, or time windows. Preserve speaker names and timestamps as metadata.

Entity-Centric Chunking

For product catalogs, customer case summaries, tickets, or knowledge articles, one entity/record can be a natural chunk.

Contextual Enrichment

Before embedding, add a compact context header describing where a passage belongs.

```
Source: Employee Handbook 2026
Section: Travel > International Travel > Reimbursement
Region: US

Chunk text: ...
```

Late Chunk Expansion

Retrieve a narrow chunk first, then fetch neighboring chunks or its parent only if the query needs more context. This can reduce context cost compared with always indexing/returning very large chunks.

Advanced Does Not Mean Better by Default

Every additional chunking mechanism must justify itself with evaluation results: better recall, better ranking, better grounded answers, or lower cost/latency.

13. OpenAI Vector Store Chunking - Current Behavior

OpenAI vector stores automatically chunk, embed, and index files added to a vector store. The API also exposes a chunking strategy when attaching files.

Current OpenAI Defaults - Verified Aug 18, 2026

The current API reference states that the **auto** file chunking strategy uses a maximum chunk size of **800 tokens** with **400 tokens of overlap**. A static strategy can customize these values. The allowed maximum chunk size is currently 100 to 4096 tokens, and overlap may not exceed half of max chunk size.

Conceptual Static Chunking Configuration

```
chunking_strategy = {
  "type": "static",
  "static": {
    "max_chunk_size_tokens": 1000,
    "chunk_overlap_tokens": 200
  }
}
```

Treat these numbers as configuration choices, not universal recommendations. The best settings should be tested against your corpus and query set.

Hosted File Search Context

OpenAI File Search is a hosted Responses API tool that searches uploaded knowledge bases using semantic and keyword search. Vector stores provide the indexed data. The number of results supplied to the model can also be limited to manage latency and context usage.

What Hosted Chunking Does Not Remove

- You still need clean source files and good document structure.
- You still need retrieval evaluation.
- You still need metadata/access-control design.
- You still need to decide whether a hosted strategy fits tables, code, contracts, or custom business records.

Architecture Choice

Use hosted vector-store chunking when it meets the product requirement. Use custom preprocessing when your documents require domain-specific boundaries, normalization, security metadata, parent-child links, or special structured extraction.

14. Practical Python Chunking Lab

The mini project compares three strategies on the same sample company policy text. The goal is to make chunking visible and testable before adding embeddings.

Sample Document

```
DOCUMENT = """
# Employee Handbook

## Vacation
Employees receive 20 vacation days per year. Unused vacation
days may carry over up to 5 days into the next calendar year.
Manager approval is required for more than 10 consecutive days.

## Expenses
Expense claims must be submitted within 30 days. Receipts are
required for expenses over $25. International travel requires
pre-approval from the department director.

## Remote Work
Employees may work remotely up to three days per week with
manager approval. Security-sensitive roles may have additional
restrictions.
"""
```

A. Simple Word-Window Chunker

```
def chunk_words(text, chunk_size=45, overlap=10):
    words = text.split()
    chunks = []
    start = 0

    while start < len(words):
        end = start + chunk_size
        chunks.append(" ".join(words[start:end]))
        if end >= len(words):
            break
        start = end - overlap

    return chunks
```

B. Heading-Aware Markdown Chunker

```
def chunk_markdown_sections(text):
    chunks = []
    current_heading = None
    current_lines = []

    for line in text.splitlines():
        if line.startswith("## "):
            if current_heading and current_lines:
                chunks.append({
                    "section": current_heading,
                    "text": "\n".join(current_lines).strip()
                })
            current_heading = line[3:].strip()
            current_lines = []
        elif line.startswith("# "):
            continue
        else:
            current_lines.append(line)

    if current_heading and current_lines:
        chunks.append({
            "section": current_heading,
            "text": "\n".join(current_lines).strip()
        })

    return chunks
```

C. Split Oversized Sections Recursively

```
def split_paragraphs(text, max_words=60):
    paragraphs = [p.strip() for p in text.split("\n\n") if p.strip()]
    output = []

    for paragraph in paragraphs:
        words = paragraph.split()
        if len(words) <= max_words:
            output.append(paragraph)
        else:
            for i in range(0, len(words), max_words):
                output.append(" ".join(words[i:i+max_words]))

    return output
```

Print and Compare

```
for i, chunk in enumerate(chunk_words(DOCUMENT), 1):
    print(f"WORD CHUNK {i}: {chunk}\n")

for item in chunk_markdown_sections(DOCUMENT):
    print(item["section"], "->", item["text"])
```

Evaluation Questions for the Lab

- Does the query "How many vacation days can carry over?" retrieve a chunk that contains both the subject and the number?
- Does "What receipt amount needs documentation?" find an expense chunk without unrelated remote-work content?
- Can each chunk be understood if displayed alone?
- Do section headings remain attached?
- How many near-duplicate chunks does overlap create?
- Would the selected chunks produce clean citations?

Mini Project Deliverable

Create a table comparing at least three chunking strategies on 15 test questions. Record expected evidence, retrieved chunk, relevance, missing context, and duplicate/noise observations.

15. Chunking Evaluation and Failure Modes

Chunking should be evaluated at the retrieval and answer levels. The right question is not "Does this chunk look good?" but "Does this strategy consistently surface sufficient evidence for real queries?"

Retrieval-Level Metrics / Checks

Check	Question
Recall@K	Does the expected evidence appear within the top K results?
Precision@K	How many of the top K results are actually relevant?
Duplicate rate	How many results are near-duplicates caused by overlap?
Context completeness	Does the retrieved unit contain enough information to interpret the fact?
Boundary errors	Was key evidence split across chunks?
Citation quality	Can the answer point to a meaningful source location?

Answer-Level Checks

- Grounded answer accuracy.
- Abstention when evidence is absent.
- Correct citation / source attribution.
- No contradiction caused by retrieving mixed policy versions.

Common Failure Modes

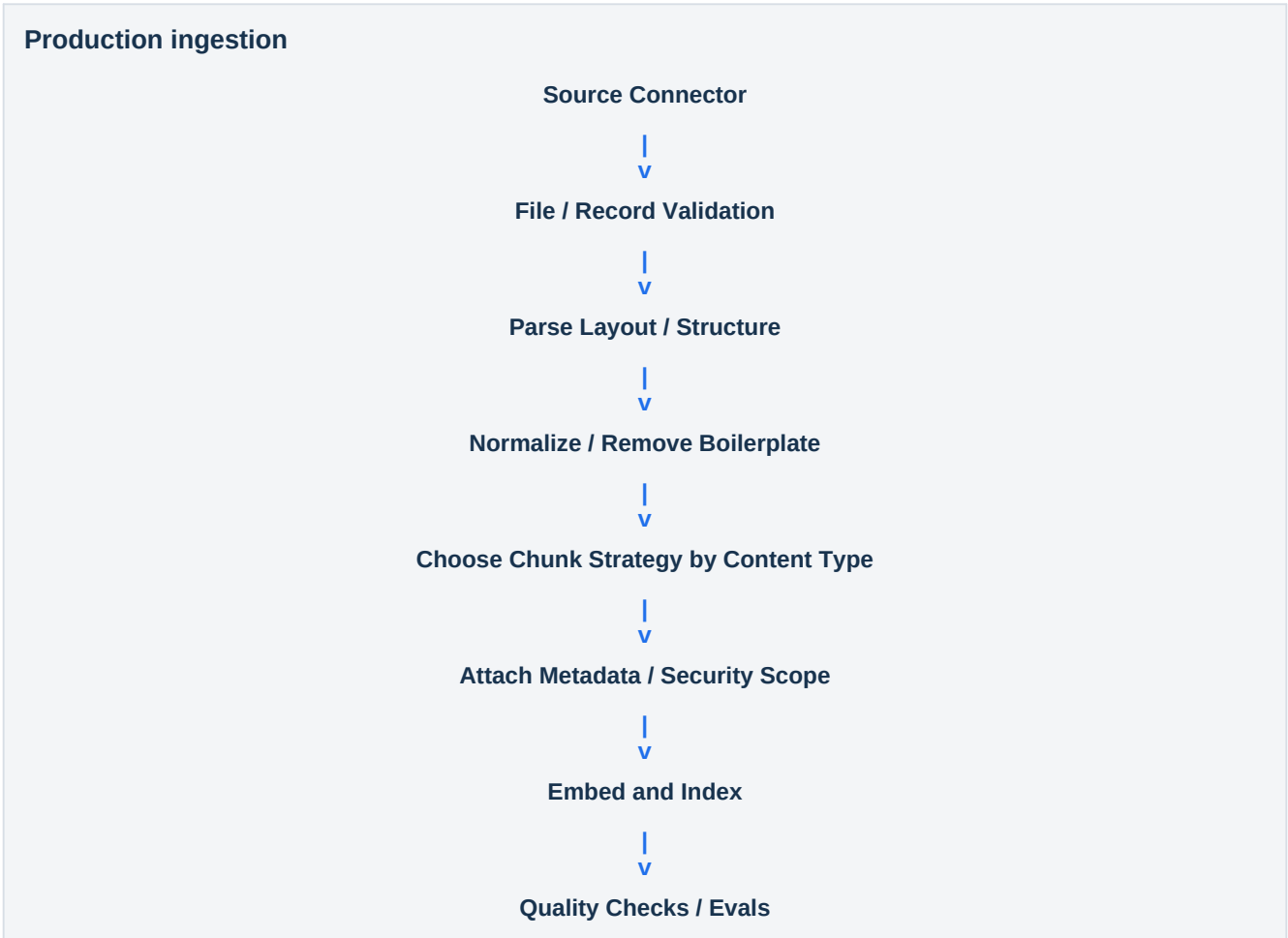
Failure	Symptom	Likely Fix
Chunk too small	Result lacks definitions/exceptions	Larger or parent expansion
Chunk too large	Lots of irrelevant context	Smaller / structural boundaries
Overlap too high	Top-K is duplicates	Reduce overlap / deduplicate
No heading context	Passage ambiguous	Prefix heading / metadata
Mixed versions	Contradictory answers	Version metadata + filter
Bad PDF parsing	Wrong reading order	Layout-aware preprocessing
Table flattened	Rows lose headers	Structured serialization
Wrong unit	Search finds pieces but cannot answer	Entity/section-specific chunks

Regression Testing

When you change chunking, re-run the complete retrieval evaluation set. A change that fixes one query can harm another section of the corpus.

16. Production Design and Business Use Cases

Production Ingestion Pipeline



Content-Type Routing

Source	Chunking Approach
Employee handbook	Heading/subsection + metadata
Support FAQs	One Q+A per chunk
CRM notes	Ticket/case or conversation segment
Product catalog	Entity/variant records
Contracts	Clause-aware + definitions/context
Source code	Function/class/module-aware
Financial tables	Structured row/record retrieval when possible
Meeting transcript	Topic or speaker-turn windows

Update Strategy

When a source document changes, production systems should re-index affected content and prevent outdated chunks from competing with current policy versions.

```
source_id: handbook
version: 2026-08
effective_date: 2026-08-01
status: active
```

Outsourcing / Client Discovery Questions

- What document types are in the knowledge base?
- Which files contain tables, forms, scans, code, or multi-column layouts?
- How do users normally ask questions?
- Which details must remain together to answer correctly?
- Are there regional or permission-specific document variants?
- How frequently do sources change?
- Do answers require page-level or clause-level citations?
- What is the current retrieval failure rate and eval dataset?

Professional Positioning

Do not sell "upload PDFs and add a chatbot." Sell a retrieval architecture: source quality, parsing, chunking, metadata, indexing, retrieval controls, evaluation, and governance.

17. Exercise, Quiz, Homework, and Interview Prep

Class Exercise A - Choose a Chunking Strategy

Content	Recommended Starting Strategy
50 FAQ questions	Q+A pairs
300-page policy manual	Heading-aware recursive
Pricing comparison table	Structured rows + headers / database
GitHub source repository	Symbol-aware code chunks
Customer support transcript	Topic/speaker windows
Legal agreement	Clause/subclause aware

Class Exercise B - Diagnose the Failure

Query: "Can unused vacation move into next year?"

Retrieved chunk:

"Up to 5 days may carry over into the next calendar year."

Problem: The retrieved sentence does not say WHAT may carry over.

Possible fix: Preserve heading/previous sentence or contextualize the chunk.

10-Question Quiz

1. What is chunking?

Answer: Dividing source content into retrievable units for embedding/indexing.

2. Why not embed every large document as one vector?

Answer: Large documents mix topics, reduce retrieval granularity, and send excessive irrelevant context.

3. What does overlap protect against?

Answer: Information being lost or separated at chunk boundaries.

4. What is a danger of too much overlap?

Answer: Duplicate retrieval and wasted storage/context.

5. What is structure-aware chunking?

Answer: Using natural source boundaries such as headings, clauses, paragraphs, rows, or symbols.

6. What is parent-child retrieval?

Answer: Search small child chunks, then return larger parent context when needed.

7. Why is metadata important?

Answer: Filtering, access control, source tracing, versioning, citations, and context.

8. Should tables always be flattened into plain paragraphs?

Answer: No. Preserve headers/rows or query structured data directly where appropriate.

9. What is the best universal chunk size?

Answer: There is none; tune with corpus-specific evaluation.

10. What should determine chunking success?

Answer: Retrieval and answer evaluation on realistic queries.

Homework Assignments

- Build the Day 18 Python chunking lab and generate chunks using at least three strategies.
- Create 20 realistic employee-handbook questions and identify the exact expected source passage for each.
- Compare a small-window strategy and a section-aware strategy. Record Recall@3 and duplicate/noise observations.
- Add metadata: source, section, page, version, department, region, and access scope.
- Design a parent-child representation for one long policy section.
- Take a table and serialize each row with its column headers so each row can be understood independently.
- Create five failure cases: too small, too large, bad overlap, missing heading, mixed versions.
- Write a one-page recommendation for a client explaining which chunk strategy you would pilot first and how you would measure success.

Interview Questions

What is chunking in RAG?

The process of dividing source content into searchable units before embedding/indexing.

How do you choose chunk size?

Start from document structure and expected questions, then tune with retrieval/answer evaluations.

Why use overlap?

To preserve context around boundaries, while balancing duplicate retrieval and storage.

Fixed vs semantic chunking?

Fixed chunking enforces predictable size; semantic chunking seeks topic-coherent boundaries.

What is parent-child retrieval?

Index small child units for precise search and expand to larger parent context for generation.

Why include headings in chunks?

They can make otherwise ambiguous passages independently understandable and semantically searchable.

How do you chunk tables?

Preserve headers and row relationships, or use structured querying when possible.

How do you evaluate chunking?

Use labeled queries, retrieval metrics, boundary/context checks, answer grounding, and regression tests.

What are common PDF chunking problems?

Wrong reading order, repeated headers, split tables, scanned text, and page boundaries that do not match semantic structure.

What is the role of metadata?

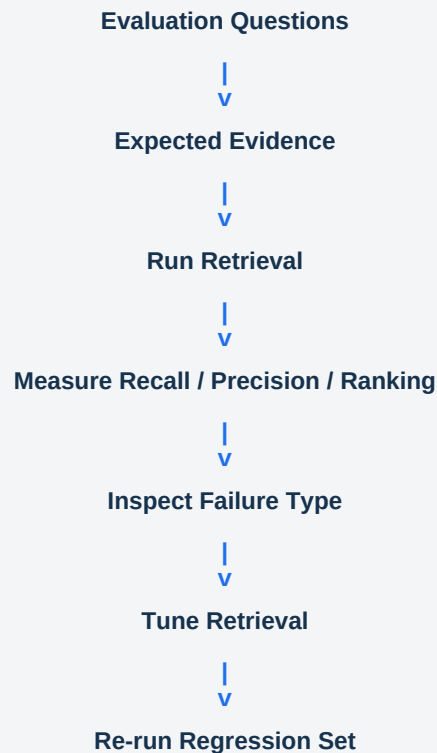
Filtering, version/security scoping, citations, source traceability, and reconstruction of context.

18. Day 19 Preview and References

Day 19 Preview - Retrieval Quality, Ranking, and RAG Evaluation

Day 19 will move from "how do we create good chunks?" to "how do we prove the retrieval system is finding the right chunks?"

Day 19 evaluation loop



Topics will include top-K selection, similarity thresholds, hybrid retrieval, metadata filters, reranking, query rewriting, retrieval metrics, answer-level groundedness, negative tests, and production monitoring.

Day 18 Final Takeaway

Chunking is where raw documents become the **searchable units of knowledge**. Good chunking preserves meaning, supports precise retrieval, carries trustworthy metadata, and produces context that the model can use without guessing.

Official OpenAI References

Current OpenAI-specific behavior below was verified on August 18, 2026.

- OpenAI Developers - Retrieval: <https://developers.openai.com/api/docs/guides/retrieval>
- OpenAI Developers - File search: <https://developers.openai.com/api/docs/guides/tools-file-search>
- OpenAI API Reference - Create vector store file:
https://developers.openai.com/api/reference/resources/vector_stores/subresources/files/methods/create
- OpenAI Developers - Embeddings: <https://developers.openai.com/api/docs/guides/embeddings>

Further Study Topics

Continue with layout-aware parsing, semantic boundary detection, parent-document retrieval, reranking/hybrid search, and domain-specific chunking for structured or regulated corpora.